

Smart MCQ Solver Challenge

Deep Learning & Generative AI Project Report

Name: Aniruddha Shukla
Roll Number: 23f2001083

1. Abstract

The Smart MCQ Solver Challenge focuses on developing a model capable of predicting the top three most probable answers for multiple-choice questions. The objective of this project was to design and evaluate different approaches for automated MCQ answering and identify the most effective solution based on the Mean Average Precision at 3 (MAP@3) evaluation metric.

The project began with exploratory data analysis (EDA) to understand the characteristics of the dataset and establish baseline expectations. Four approaches were then implemented and compared. Models were evaluated using Accuracy and MAP@3 to compare their effectiveness in ranking the correct answers.

Four models were tested: TF-IDF with CatBoost, a Transformer trained from scratch, SciBERT, and DeBERTa-v3-base. DeBERTa achieved the best validation MAP@3 of 0.9988 and was used for the final Kaggle submission. However, the Kaggle score was much lower than the validation score because many questions appeared in both the training and test sets.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge requires participants to develop an intelligent system capable of automatically solving multiple-choice questions. Each question consists of a prompt and five answer options labelled A, B, C, D, and E. Instead of predicting only the correct answer, the system must rank the top three most probable answers in descending order of confidence.

2.2 Project Objective

The primary objective of this project was to design and evaluate different NLP approaches for automated multiple-choice question answering. The project involved performing exploratory data analysis to understand the dataset, implementing a classical machine learning baseline using TF-IDF features, training a transformer encoder from scratch, fine-tuning SciBERT and DeBERTa-v3-base for contextual language understanding, and comparing all approaches using Accuracy and MAP@3 to identify the most effective solution for the competition.

2.3 Report Structure

The remainder of this report is organized as follows. Section 3 describes the dataset, preprocessing steps, and exploratory data analysis. Section 4 explains the tokenization strategy adopted across the different models. Section 5 presents the four implemented models and the experimental methodology behind each. Section 6 compares the performance of all models using appropriate evaluation metrics and discusses the validation-versus-leaderboard gap. Section 7 concludes the report by summarizing key findings, challenges faced, and potential future improvements.

3. Dataset and Preprocessing

3.1 Dataset Description

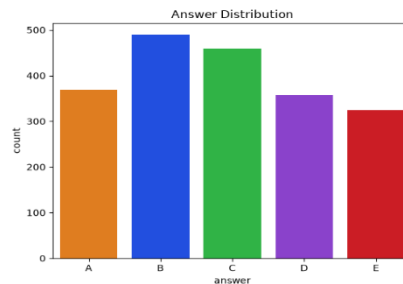
The dataset used in this project was provided as part of the Smart MCQ Solver Challenge. It consists of a training dataset containing 2,000 multiple-choice questions and a test dataset containing 500 questions. Each question includes

a prompt and five answer options labelled A, B, C, D, and E. The training dataset also contains the correct answer label, while the test dataset is unlabelled and used for leaderboard evaluation. Neither dataset contains missing values, and there are no exact duplicate rows in either file.

3.2 Exploratory Data Analysis

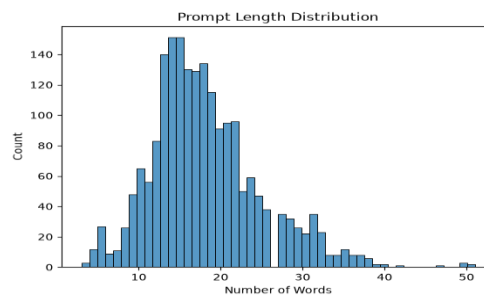
EDA was performed to understand the data and identify patterns that could influence model development. A separate, more detailed EDA notebook (EDA.ipynb) is also there; the key findings are summarized below.

3.2.1 Correct Answer Distribution



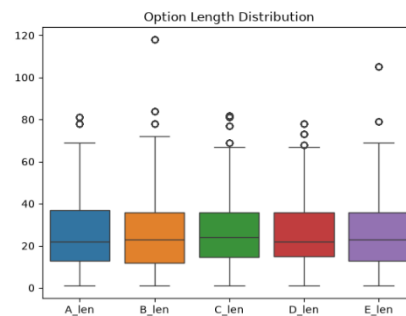
The five answer classes (A-E) are reasonably well distributed, although classes B and C occur more frequently than the others, while E is the least common (B: 24.5%, C: 23.0%, A: 18.5%, D: 17.9%, E: 16.2%). The imbalance wasn't severe enough for resampling, so a stratified split was used to keep the class proportions consistent between train and validation.

3.2.2 Distribution of Question (Prompt) Length



The prompts contain an average of 18 words, with the shortest prompt consisting of 3 words and the longest containing 51 words. Most questions fall within the range of 12-22 words, indicating that the dataset primarily consists of short to medium length textual inputs.

3.2.3 Option Length Distribution



The distribution of answer option lengths across all five choices (A-E) shows a median of 22-24 words and an average of approximately 26 words. While most answer options fall within a similar range, a small number of outliers contain over 100 words. Always selecting the longest option as the answer - was correct only 34.95% of the time.

This and prompt length analysis was useful in selecting an appropriate maximum sequence length during tokenization, since the majority of questions are relatively short, a moderate sequence length (128 - 256 tokens) was sufficient.

3.2.4 Prompt Repetition and Train/Test Overlap

An analysis of question prompts revealed 242 repeated prompts within the training dataset. All repeated prompts were associated with the same correct answer, indicating no conflicting labels among duplicates. 8 duplicate prompts within the test dataset were also present.

A comparison between the training and test datasets identified 267 common prompts, representing approximately 54.3% of the 492 unique test prompts. Among these common prompts, 95.5% also had identical answer options, meaning over half of the test set had basically already been seen during training.

3.3 Data Preprocessing

For the classical machine learning models, the question prompt was concatenated with each answer option to form a single text sequence, which was then vectorized using TF-IDF (word-level and character-level n-grams). This representation allowed the models to capture relationships between the question and its candidate answers.

For the transformer-based models (the custom encoder, SciBERT, and DeBERTa-v3-base), each question prompt was paired individually with each answer option and processed using the respective model's tokenizer, with padding and truncation applied to produce fixed-length sequences.

3.4 Data Augmentation

No data augmentation techniques were applied in this project.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

The best-performing model in this project was the **fine-tuned DeBERTa-v3-base model** (microsoft/deberta-v3-base), which comes with a SentencePiece-based subword tokenizer with a vocabulary of approximately 128,000 tokens. Unlike SciBERT's WordPiece tokenizer, DeBERTa-v3's tokenizer is trained directly on a large, general-domain corpus.

4.2 Justification

SentencePiece tokenization, which comes with the DeBERTa-v3-base model, treats the input as a raw character stream and learns subword units directly from data, which makes it more robust to unusual punctuation, numbers, and rare technical terms than whitespace-dependent tokenizers.

4.3 Implementation Details

Tokenization was implemented using the Hugging Face **AutoTokenizer** library. Each question prompt was paired individually with each of the five answer options, producing five separate input sequences per question. The tokenizer applied truncation and padding to a fixed maximum sequence length of 256 tokens (128 max sequence length for scratch model due to memory limits), ensuring uniform input dimensions for efficient batch processing. The tokenized inputs were organized using custom PyTorch Dataset and DataLoader classes, generating batches of input IDs and attention masks for training and inference. The same pairing strategy (prompt + single option per sequence) was used consistently across the custom transformer, SciBERT, and DeBERTa-v3 models to keep the modeling approach comparable.

4.4 All Tokenizer Comparison

Four different text-representation strategies are compared based on the models used:

- Custom word-level tokenizer: a vocabulary built directly from the training corpus (minimum frequency = 2), producing a compact 3,190-token vocabulary with no subword handling.
- WordPiece tokenization via SciBERT: subword tokenization using a vocabulary pretrained on scientific literature, matched to the technical vocabulary observed in the dataset.
- SentencePiece tokenization via DeBERTa-v3: a larger, general-domain subword vocabulary combined with disentangled attention.
- TF-IDF word and character n-grams: included as a baseline to test a simple baseline score.

5. Modelling and Experimentation

Four models were developed and compared, presented below in increasing order of complexity: a classical machine learning model, a transformer encoder trained from scratch, a fine-tuned SciBERT model, and a fine-tuned DeBERTa-v3-base model.

5.1 Model 1: TF-IDF + CatBoost

Architecture / Approach: A classical pipeline was built using TF-IDF features over (question, option) pairs, framed as a binary classification problem. Each question-option pair was vectorized using a combination of word-level (1-3 n-grams, up to 120,000 features) and character-level (3-5 n-grams, up to 90,000 features) TF-IDF representations.

The vectorized features were used to train a CatBoost classifier, with the predicted probability used as the ranking score for each option, and the top-3 highest-scoring options were selected as prediction

Salient Points

- Computationally inexpensive to train and run compared to transformer fine-tuning.
- Relies entirely on lexical overlap and word/character statistics - no contextual understanding of language.

Results:

CatBoost achieved Validation Accuracy = 0.990 and MAP@3 = 0.996. On Kaggle leaderboard, the classical ML model gave a score of 0.747

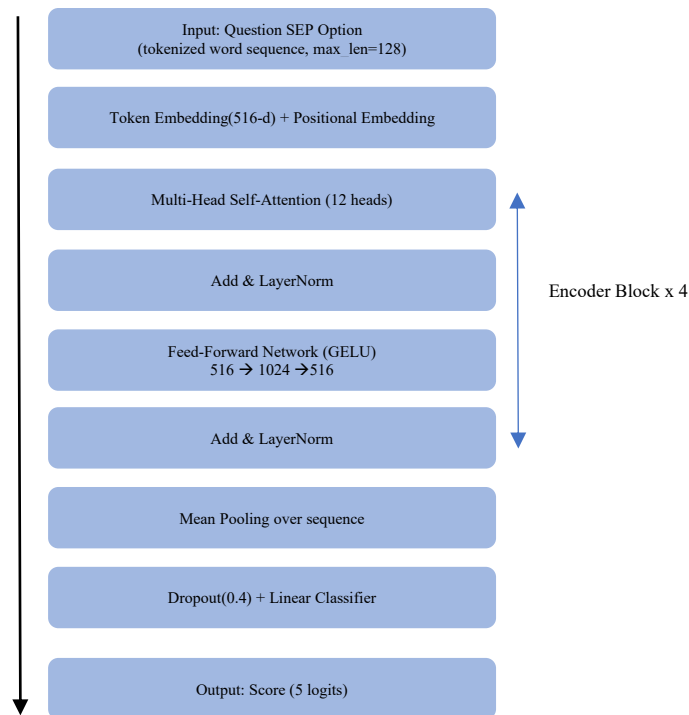
5.2 Model 2: Transformer Encoder with Multi-Head Self-Attention (Trained from Scratch)

A custom Transformer encoder was implemented from scratch using PyTorch to better understand the internal workings of transformer-based models, and to establish a baseline for how well the task can be learned purely from this dataset without any pretrained language knowledge. All model parameters were learned from scratch using only the project's training data. The model processes each question-option pair independently and predicts a score for each of the five answer choices.

Architecture

- Embedding Layer: learns token embeddings and positional embeddings to capture both word meaning and token order.
- Multi-Head Self-Attention: employs 12 attention heads to capture relationships between different parts of the input sequence.
- Transformer Encoder: consists of 4 stacked encoder blocks, each containing multi-head attention, residual connections, layer normalization, and a feed-forward network with GELU activation.

- Pooling & Classification: mean pooling aggregates sequence representations, followed by dropout and a linear layer producing one score per answer option.



Hyperparameters

Parameter	Value	Reason
Embedding Dimension	516	Tuned multiple times but this value finally provided better results
Attention Heads	12	Allows the model to learn multiple semantic relationships simultaneously (43 dimensions per head).
Encoder Layers	4	Kept small size to prevent overfitting on small dataset.
Feed-Forward Dimension	1024	Two times the embedding size, giving the network sufficient capacity to model token interactions.
Dropout	0.4	Higher than normal to reduce overfitting on small dataset
Max Sequence Length	128	Chosen based on EDA and computation limits on limited 4GB GPU.
Optimizer	AdamW	Usually the default choice for training from scratch
Loss Function	Cross-Entropy Loss	Standard for multi-class classification across the five answer choices.
Learning Rate	3e-4	Tuned several times, and then this value gave the best results and was used for final run
LR Scheduler	Cosine Annealing	Gradually decreases the learning rate to promote larger parameter updates initially and finer adjustments as training converges.
Batch Size	16	Largest batch size that could fit in the 4GB VRAM.

5.3 Model 3: Fine-Tuned SciBERT

Unlike BERT, which is pretrained on general-domain corpora such as Wikipedia and BookCorpus, SciBERT is pretrained on a large corpus of scientific publications using a domain-specific SciVocab WordPiece vocabulary. The model adopts an encoder-only Transformer architecture consisting of 12 Transformer encoder layers, each with 12 multi-head self-attention heads, a hidden size of 768, and an intermediate feed-forward dimension of 3072. The model contains approximately 110 million parameters.

Salient Points

- Uses bidirectional self-attention to capture contextual relationships between words.
- Pretrained on scientific literature, giving better understanding of technical vocabulary than general-purpose model.
- Uses WordPiece tokenization with a scientific vocabulary (SciVocab), reducing out-of-vocabulary words for domain-specific text.

Fine-Tuning Strategy

The pretrained SciBERT encoder was fine-tuned end-to-end (i.e., no layers were frozen). The contextual token representations produced by the encoder were aggregated using masked mean pooling over the final hidden states, followed by a linear classification head that produced one score for each answer option. The model was optimized using the AdamW optimizer and Cross-Entropy Loss, with a learning rate of 2×10^{-5} , a batch size of 4, and trained for 3 epochs.

5.4 Model 4: Fine-Tuned DeBERTa-v3-base (Best-Performing Model).

Architecture

DeBERTa-v3-base was tried along with SciBERT. It uses 12 transformer layers with a hidden size of 768. It uses a different attention mechanism, disentangled attention mechanism, where each token is represented by two separate vectors encoding content and relative position, with attention weights computed using both. It also uses an ELECTRA-style pretraining objective instead of standard masked language modeling.

Fine-Tuning Strategy

The full DeBERTa-v3-base encoder was finetuned with a masked mean-pooled classification head, dropout = 0.3, using the AdamW optimizer (learning rate = $1e-5$, weight decay = 0.01) with a linear warmup-and-decay schedule (10% warmup steps). Training was run for 6 epochs with a batch size of 4 (small batch size due to OOM error along with 256 sequence length), and the checkpoint with the highest validation MAP@3 was retained as the final model.

This model was selected as the final submission for the Kaggle leaderboard, based on it achieving the highest validation MAP@3 among all four approaches evaluated.

6. Performance and Comparative Analysis

6.1 Evaluation Metrics

Following metrics were used throughout this project. Accuracy measures the proportion of questions for which the single top-ranked prediction matches the correct answer. Loss was tracked to measure convergence. Mean Average Precision at 3 (MAP@3), the competition's primary metric, rewards the model for ranking the correct answer within its top 3 predictions, weighted by rank: a score of 1.0 if correct at rank 1, 0.5 at rank 2, and 1/3 at rank 3.

6.2 WandB Runs

All WandB runs are logged to project: 23f2001083-t22026

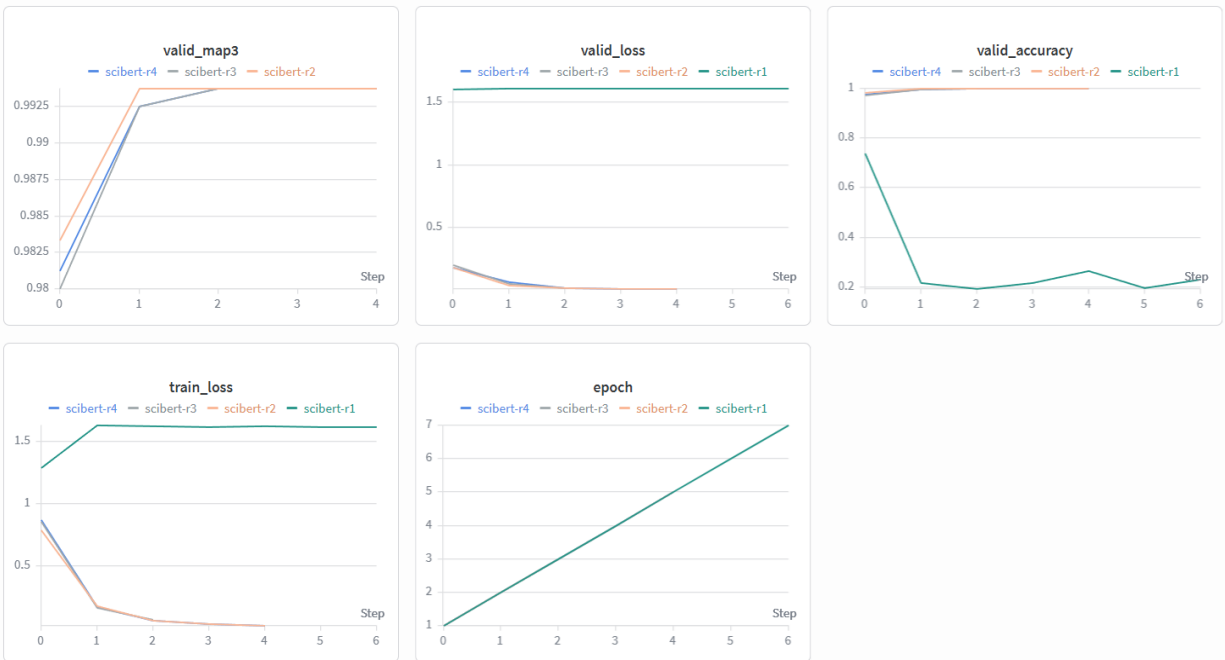
6.2.1 Transformer Encoder from scratch

Multiple configurations were tested, varying model size (128–516 embedding dim, 2–4 layers), dropout, learning rate ($2e-5$ – $3e-4$), and sequence length(128-256). Learning rate proved to be the dominant factor: even the smallest model converged well at $LR = 3e-4$ ($MAP@3 > 0.9$). The final configuration (embedding dim = 516, 4 layers, 12 heads, dropout = 0.4, $LR = 3e-4$, 50 epochs) was chosen for its strong and stable convergence.



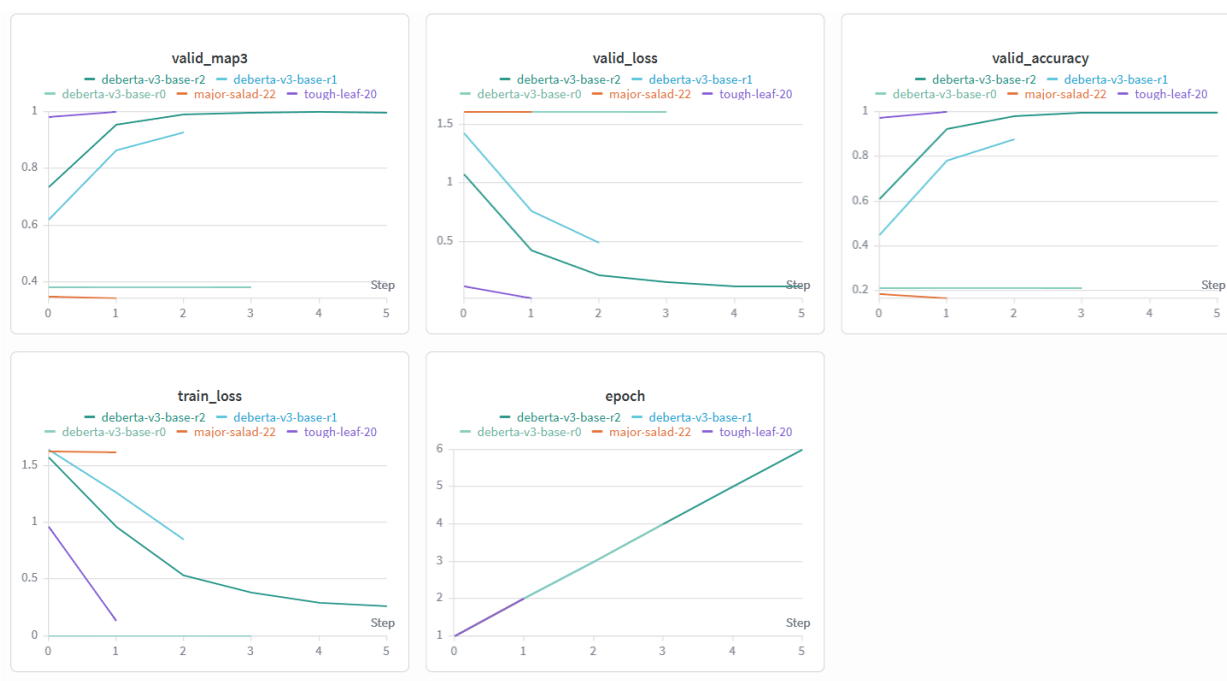
6.2.2 Fine-Tuned SciBERT

SciBERT was finetuned at batch size 4 and max sequence length 256. R2–R4 converged within the first 2–3 epochs to near-ceiling validation $MAP@3$ (~ 0.99).



6.2.3 Fine-Tuned DeBERTa-v3-base

Run r1 reached $MAP@3 = 0.928$ in 3 epochs. Learning rate was reduced from $2e-5$ to $1e-5$ for run r2, which achieved the best overall result of this project: Validation Accuracy = 0.9975 and $MAP@3 = 0.9988$. This was selected as the final submitted model.



6.3 Comparative Summary

Model	Approach	Val. Accuracy	Val. MAP@3
1. Classical ML	TF-IDF + CatBoost	0.99	0.9958
2. Scratch Transformer	Custom encoder, no pretraining	0.9663	0.9768
3. SciBERT	Finetuned, domain-specific pretrained	1.000	0.9938
4. DeBERTa-v3-base	Finetuned, general-purpose pretrained	0.9975	0.9988

6.4 Validation Score vs. Kaggle Leaderboard Gap

Upon comparing the validation scores above to the actual Kaggle leaderboard scores, every model scored below 0.753 on the leaderboard, while the top-ranked competitor on the leaderboard scored only 0.769.

This gap is explained by the train/test overlap identified during EDA. 54.3% of test prompts are exact duplicates of training prompts; a model that has memorized these duplicates can score close to 1.0 on that portion, but must rely on genuine generalization for the remaining 45.7% of novel test questions.

6.5 Kaggle Leaderboard Performance

Currently on rank 402 with a score of 0.75270 on public leaderboard on Kaggle. This score was achieved by the finetuned DeBERTa-v3-base model.

7. Conclusion and Future Work

7.1 Key Learnings

- Changing the learning rate had a noticeable effect on the scratch Transformer. Increasing it to $3e-4$ gave much faster convergence than the smaller learning rates tested

- In this dataset, validation MAP@3 was not a reliable indicator of Kaggle performance because many prompts were duplicated between the training and test sets.

7.2 Challenges Faced

- Diagnosing why validation MAP@3 scores were near-ceiling for multiple unrelated model types (transformers and classical ML alike) required a dedicated EDA investigation into prompt duplication, which was not obvious from model training curves alone.
- Balancing GPU memory constraints against sequence length and batch size, particularly for DeBERTa-v3-base, required using a smaller batch size (4) than for the other models. Also the scratch implementation of transformer encoder was done on limited 4GB GPU which also led to out of memory error when increased sequence length from 128 to 256.

7.3 Areas for Improvement / Future Work

- Group exact-duplicate prompts before performing the train/validation split so that validation scores are not inflated by memorized duplicates, giving a more honest estimate of generalization performance.
- Experiment with data augmentation (e.g., paraphrasing or option shuffling) to increase effective training diversity beyond the current 2,000 questions.
- Exploring larger or more specialized pretrained models (e.g., DeBERTa-v3-large) and proper ensembling across various models, rather than treating them as independent submissions.
- Implementing RAG based solution for better results. Since the data is domain specific, RAG can be used to provide context to the model before predicting the options.

8. References

1. Vaswani, A. et al., "Attention Is All You Need," NeurIPS, 2017.
2. Beltagy et al., "SciBERT: A Pretrained Language Model for Scientific Text," EMNLP-IJCNLP, 2019.
3. He et al., "DeBERTa: Decoding-enhanced BERT with Disentangled Attention," ICLR, 2021.
4. Hugging Face, SciBERT Model Card: https://huggingface.co/allenai/scibert_scivocab_uncased
5. Hugging Face, DeBERTa-v3-base Model Card: <https://huggingface.co/microsoft/deberta-v3-base>
6. PyTorch: <https://pytorch.org>
7. Weights & Biases: <https://wandb.ai>